Data Engineering

MINI PROJECT

Weather Data Pipeline

🚀 Mini Project: Weather Data Pipeline with Python

📌 Project Title

Build a Simple Data Engineering Pipeline Using a Public API

🎯 Project Objective

Build a complete ETL (Extract, Transform, Load) pipeline that:

- Extracts data from a Public API
- Loads data into a Pandas DataFrame
- Cleans and validates the data
- Performs transformations and feature engineering
- Generates summary statistics
- Filters and sorts records
- Saves cleaned data into CSV and JSON
- Produces a simple analytical report

🏢 Business Scenario

A travel and tourism company wants to monitor weather conditions across major African cities to help customers plan their trips.

As a Data Engineer, you need to:

1. Collect weather data from multiple cities.
2. Clean and standardize the data.
3. Generate useful insights.
4. Store the processed data for reporting.

📁 Project Structure

```
weather_project/
|
```

```
├── weather_pipeline.ipynb
├── raw_weather.csv
├── cleaned_weather.csv
├── weather_report.csv
├── weather_data.json
└── README.md
```

🛠 Tools Required

- Python
- Jupyter Notebook
- Pandas
- Requests
- JSON Library

Install dependencies:

pip install pandas requests

📊 Data Source

Use the Open-Meteo API (Free, No API Key Required)

Example:

https://api.open-meteo.com/v1/forecast?latitude=9.03&longitude=38.74&current=temperature_2m,relative_humidity_2m,wind_speed_10m

---

🔄 ETL Workflow

Step 1: Extract Data from API

```
import requests
import pandas as pd

cities = {
    "Addis Ababa": (9.03, 38.74),
```

```
    "Nairobi": (-1.29, 36.82),
    "Cairo": (30.04, 31.24),
    "Lagos": (6.52, 3.37),
    "Johannesburg": (-26.20, 28.04)
}

weather_records = []

for city, coords in cities.items():

    lat, lon = coords

    url = f"https://api.open-
meteo.com/v1/forecast?latitude={lat}&longitude={lon}&current=tempera
ture_2m,relative_humidity_2m,wind_speed_10m"

    response = requests.get(url)

    data = response.json()

    weather = data["current"]

    weather_records.append({
        "City": city,
        "Temperature": weather["temperature_2m"],
        "Humidity": weather["relative_humidity_2m"],
        "WindSpeed": weather["wind_speed_10m"],
        "DateTime": weather["time"]
    })

df = pd.DataFrame(weather_records)

df
```

Step 2: Explore the Dataset

View First Rows

df.head()

View Last Rows

df.tail()

Shape

df.shape

Column Names

df.columns

Data Types

df.dtypes

Dataset Information

df.info()

Statistical Summary

df.describe()

Step 3: Create Intentional Data Issues

(Add for Practice)

df.loc[1, "Temperature"] = None
df.loc[3, "Humidity"] = None

Step 4: Handle Missing Values

Check Missing Values

df.isnull().sum()

Fill Missing Numeric Values with Mean

```
df["Temperature"] = df["Temperature"].fillna(
    df["Temperature"].mean()
)

df["Humidity"] = df["Humidity"].fillna(
    df["Humidity"].mean()
)
```

Verify

```
df.isnull().sum()
```

Step 5: Remove Duplicate Records

Create a duplicate:

```
df = pd.concat([df, df.iloc[[0]]])
```

Check duplicates:

```
df.duplicated().sum()
```

Remove duplicates:

```
df = df.drop_duplicates()
```

Step 6: Rename Columns

```
df.rename(columns={
    "Temperature":"Temperature_C",
    "Humidity":"Humidity_Percent",
    "WindSpeed":"WindSpeed_kmh"
}, inplace=True)
```

Result:

```
df.head()
```

Step 7: Convert Data Types

```
df["DateTime"] = pd.to_datetime(df["DateTime"])
```

Verify:

```
df.dtypes
```

Step 8: Feature Engineering

Create Temperature in Fahrenheit

```
df["Temperature_F"] = (
    df["Temperature_C"] * 9/5
) + 32
```

Extract Date

```
df["Date"] = df["DateTime"].dt.date
```

Extract Hour

```
df["Hour"] = df["DateTime"].dt.hour
```

Extract Day Name

```
df["Day_Name"] = df["DateTime"].dt.day_name()
```

Step 9: Filtering Data

Cities hotter than 25˚C

```
hot_cities = df[
    df["Temperature_C"] > 25
]

hot_cities
```

Step 10: Sorting Data

Highest temperature first:

```
df.sort_values(
    by="Temperature_C",
    ascending=False
)
```

Step 11: Grouping and Aggregation

Average weather statistics

```
df.groupby("City").agg({
    "Temperature_C":"mean",
    "Humidity_Percent":"mean",
    "WindSpeed_kmh":"mean"
})
```

Step 12: Create Categories

```
def weather_category(temp):

    if temp > 30:
        return "Hot"

    elif temp > 20:
        return "Warm"

    else:
        return "Cold"

df["Weather_Category"] = df[
    "Temperature_C"
].apply(weather_category)
```

Output:

df[["City","Temperature_C","Weather_Category"]]

Step 13: Query Data

df.query("Temperature_C > 20")

Step 14: Value Counts

df["Weather_Category"].value_counts()

Step 15: Calculate Additional Metrics

Temperature Range

```
max_temp = df["Temperature_C"].max()
min_temp = df["Temperature_C"].min()

print("Max:", max_temp)
print("Min:", min_temp)
```

Average Temperature

df["Temperature_C"].mean()

Median Temperature

df["Temperature_C"].median()

Step 16: Generate Summary Report

```
report = pd.DataFrame({
    "Metric": [
        "Average Temp",
        "Max Temp",
        "Min Temp",
        "Average Humidity"
```

```
    ],
    "Value": [
        df["Temperature_C"].mean(),
        df["Temperature_C"].max(),
        df["Temperature_C"].min(),
        df["Humidity_Percent"].mean()
    ]
})

report
```

Step 17: Save Data

Save Raw Data

```
df.to_csv(
    "raw_weather.csv",
    index=False
)
```

Save Cleaned Data

```
df.to_csv(
    "cleaned_weather.csv",
    index=False
)
```

Save JSON

```
df.to_json(
    "weather_data.json",
    orient="records",
    indent=4
)
```

Save Report

```
report.to_csv(
    "weather_report.csv",
    index=False
)
```

This project gives students hands-on experience with the Pandas operations that Data Engineers use daily when preparing data for data warehouses, dashboards, and analytics platforms.